

## COMP364 Quiz 3

Questions ??–?? will not be tested in the in-class quiz or in the midterm exams. They are included here because they are useful for completing the homework.

### Question 1. Logistic function (2 points)

State the logistic function, its derivative in two forms, and its inverse (the logit function).

**Solution.**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

$$\sigma^{-1}(y) = \log\left(\frac{y}{1 - y}\right)$$

**Explanation.**

**Derivative.** Using the chain rule,

$$\begin{aligned}\sigma'(x) &= \frac{d}{dx}(1 + e^{-x})^{-1} \\ &= -(1 + e^{-x})^{-2}(-e^{-x}) \\ &= \frac{e^{-x}}{(1 + e^{-x})^2}.\end{aligned}$$

Since

$$1 - \sigma(x) = 1 - \frac{1}{1 + e^{-x}} = \frac{e^{-x}}{1 + e^{-x}},$$

we can also write

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

**Inverse.**

Let  $y = \sigma(x)$ . Then

$$\begin{aligned} y &= \frac{1}{1 + e^{-x}}, \\ 1 + e^{-x} &= \frac{1}{y}, \\ e^{-x} &= \frac{1 - y}{y}, \\ e^x &= \frac{y}{1 - y}. \end{aligned}$$

Taking the natural logarithm of both sides gives

$$x = \log\left(\frac{y}{1 - y}\right),$$

so

$$\sigma^{-1}(y) = \log\left(\frac{y}{1 - y}\right).$$

### Question 2. Numerically stable logistic function (2 points)

Give some pseudo-code that computes a numerically stable version of the logistic function, avoiding large positive exponents that could overflow.

**Solution.**

```

if  $x \geq 0$  then
    return  $\frac{1}{1 + e^{-x}}$ 
else
    return  $\frac{e^x}{1 + e^x}$ 

```

**Explanation.**

When  $x \geq 0$ , the expression uses  $e^{-x}$ , whose exponent is nonpositive; when  $x < 0$ , it uses  $e^x$ , whose exponent is also negative. Thus the pseudocode avoids computing a potentially overflowing large positive exponential while producing the same value as the logistic function.

### Question 3. Gradient descent optimization (5 points)

Let  $L(w)$  be a loss function that we wish to minimize with respect to the parameter  $w$  by making small changes to  $w$  until we get close to the minimum. The size of the updates will be controlled by a small number  $\lambda > 0$ , called the *learning rate*. Assume that we know the derivative of the loss function,  $L'(w)$ . State the basic principle of gradient descent optimization that should be applied in this situation.

**Solution.**

Repeatedly update  $w$  according to

$$w_{\text{new}} = w_{\text{old}} - \lambda L'(w_{\text{old}}).$$

**Explanation.**

If the derivative (“gradient”) of  $L$  is positive, then increasing  $w$  would also increase the loss  $L$ . But we want to *decrease* the loss (i.e., “descend”), so we should *decrease*  $w$ . That explains the minus sign in the above update formula. If the derivative of  $L$  is negative, then we need to *increase*  $w$ , and the minus sign again makes sense.

The learning rate  $\lambda$  needs to be small, because if it is too big then we might overshoot the minimum and increase the loss instead.

**Question 4. Derivative and updates of one-dimensional unit with cross-entropy loss**

Suppose a single-unit neural network has a single real-valued input  $x$ . It computes output probability  $y$  by first computing a logit  $z$ :

$$z = wx + b,$$

where  $w$  is a real-valued weight and  $b$  is a real-valued bias. Then output probability  $y$  is given by

$$y = \sigma(z)$$

The neural network is a binary classifier, whose ground-truth label  $t$  is either 0 or 1. The network is trained using binary cross-entropy loss.

(a) **(2 points)** Write down the loss function  $L$  as a function of  $y$  and  $t$ .

**Solution.**

$$L = -(t \log y + (1 - t) \log(1 - y)).$$

**Explanation.**

You can take this as the definition of binary cross-entropy loss, which we do not study in any further detail in this course. However, note that there is a connection to entropy here. The entropy of the probability distribution  $[t, 1 - t]$  is  $-(t \log t + (1 - t) \log(1 - t))$ . The entropy of the probability distribution  $[y, 1 - y]$  is  $-(y \log y + (1 - y) \log(1 - y))$ . The *cross*-entropy has the same form but it uses  $t$  outside the log functions and  $y$  inside the log functions.

(b) **(2 points)** Write down  $\frac{dL}{dz}$ , the derivative of  $L$  with respect to the logit.

**Solution.**

$$\frac{dL}{dz} = y - t.$$

**Explanation.**

Using the chain rule,

$$\frac{dL}{dz} = \frac{dL}{dy} \frac{dy}{dz}.$$

From

$$L = -(t \log y + (1 - t) \log(1 - y)),$$

we get

$$\frac{dL}{dy} = -\frac{t}{y} + \frac{1 - t}{1 - y}.$$

Since  $y = \sigma(z)$ , we have

$$\frac{dy}{dz} = y(1 - y).$$

Therefore,

$$\begin{aligned} \frac{dL}{dz} &= \left( -\frac{t}{y} + \frac{1 - t}{1 - y} \right) y(1 - y) \\ &= -t(1 - y) + (1 - t)y \\ &= y - t. \end{aligned}$$

(c) **(2 points)** Write down the updates for  $w$  and  $b$  assuming gradient descent optimization with learning rate  $\lambda$ .

**Solution.**

$$\begin{aligned} w &= w - \lambda x(y - t) \\ b &= b - \lambda(y - t) \end{aligned}$$

**Explanation.**

These updates apply the gradient-descent rule to each parameter using the loss derivative with respect to the logit,  $\frac{dL}{dz} = y - t$ . Changing  $w$  changes  $z$  by a factor of  $x$ , while changing  $b$  changes  $z$  directly, which gives the factors  $x(y - t)$  and  $y - t$  in the two updates.

(d) **(2 points)** Let's generalize this to the case of two weights and two inputs to the neuron. So  $z = w_1 x_1 + w_2 x_2 + b$ . Give the updates for  $w_1, w_2$  and  $b$  assuming gradient descent optimization with learning rate  $\lambda$ .

**Solution.**

$$\begin{aligned} w_1 &= w_1 - \lambda x_1(y - t) \\ w_2 &= w_2 - \lambda x_2(y - t) \\ b &= b - \lambda(y - t) \end{aligned}$$

**Explanation.**

This is a straightforward generalization.

**Question 5. Softmax definition**

(a) **(2 points)** Give a definition of the softmax function for logits  $x_0, x_1, \dots, x_{n-1}$ .

**Solution.**

$$\text{softmax}(x_0, x_1, \dots, x_{n-1}) = \frac{1}{S}(e^{x_0}, e^{x_1}, \dots, e^{x_{n-1}}),$$

where  $S$  is a constant ensuring the resulting values sum to 1:

$$S = \sum_{i=0}^{n-1} e^{x_i}.$$

**Explanation.**

The softmax is a generalization of the sigmoid, because  $\text{softmax}(x, 0) = (\sigma(x), 1 - \sigma(x))$ .

(b) **(2 points)** Explain how to produce normalized logits from the probabilities  $p_0, p_1, \dots, p_{n-1}$

**Solution.**

The unnormalized logits are  $x_i = \log(p_i)$ . To normalize, subtract the largest logit from all logits, so that the maximum logit has the value 0 and the rest are negative.

**Explanation.**

This is just the inverse of the softmax function, but with a particular choice of additive normalization. Ensuring that logits are all nonnegative means that the softmax will be numerically stable, because it will not exponentiate large values.

**Question 6. Converting between logits and probabilities**

A neural network computes its three final outputs using a softmax layer.

(a) **(5 points)** The output probabilities are (0.21, 0.66, 0.13). What were the normalized logits?

**Solution.**

(-1.145, 0, -1.625)

**Explanation.**

The unnormalized logits are  $(\log(0.21), \log(0.66), \log(0.13))$ . We normalize by subtracting the largest,  $\log(0.66)$ , obtaining

$$\begin{aligned} &(\log(0.21) - \log(0.66), \log(0.66) - \log(0.66), \log(0.13) - \log(0.66)) \\ &\approx (-1.145, 0, -1.625). \end{aligned}$$

(b) **(5 points)** The (unnormalized) logits are  $(-1.1, 3.1, 3.7)$ . What are the output probabilities?

**Solution.**

$(0.005, 0.352, 0.642)$

**Explanation.**

$$\mathbf{p} = \frac{(e^{-1.1}, e^{3.1}, e^{3.7})}{e^{-1.1} + e^{3.1} + e^{3.7}} \approx (0.005, 0.352, 0.642)$$

### Question 7. Parameter count in fully connected network (5 points)

A neural network takes 20 inputs and classifies instances into 8 categories. It has one fully-connected hidden layer containing 10 units. The hidden layer connects to an output layer of 8 units, which uses a softmax activation function to compute class probabilities. Assuming every layer includes both weights and bias parameters, what is the total number of trainable parameters in the network?

**Solution.**

The hidden layer has  $20 \times 10$  weights and 10 biases. The output layer has  $10 \times 8$  weights and 8 biases. Summing these gives 298 parameters.

*In the next few questions you may assume that your Python code includes the lines*

```
1 import torch
2 import torch.nn as nn
```

### Question 8. Creating a single neuron and linear layer in PyTorch (4 points)

Give Python code that would be inserted into the `__init__()` constructor of a class derived from `nn.Module` to create (a) a single linear unit with bias and with 7 inputs; and (b) a layer of 12 linear units with bias, each having 8 inputs.

**Solution.**

```

1      # (a)
2      self.neuron = nn.Linear(7, 1)
3      # (b)
4      self.layer = nn.Linear(8, 12)

```

**Question 9. Training via a PyTorch optimizer (4 points)**

Fill in the three missing lines of the following training loop:

```

1      model = SomeModel() # derived from nn.Module
2      criterion = nn.SomeLossFunction()
3      optimizer = torch.optim.SomeOptimizer(model.parameters(), lr=1e-3)
4      model.train() # put into training mode
5      for x, labels in training_data:
6          outputs = model(x)
7          loss = criterion(outputs, labels)
8          -----
9          -----
10         -----

```

**Solution.**

```

1      optimizer.zero_grad()
2      loss.backward()
3      optimizer.step()

```

**Question 10. ReLU (4 points)**

Let  $r(x)$  be the ReLU function. Then what is (a)  $r(-4)$ ; (b)  $r(41)$ ?

**Solution.**

(a) 0; (b) 41.

(c) Fill in the missing code in this snippet from the `__init__()` constructor of a PyTorch model, adding a ReLU layer after each linear layer.

```

1      self.relu = nn.ReLU()
2      self.layers = nn.ModuleList()
3      for _ in range(3):
4          self.layers.append(nn.Linear(12, 12))
5          -----

```

**Solution.**

```
self.layers.append(self.relu)
```

**Question 11. Cosine similarity (10 points)**

Compute the cosine similarity of each pair of vectors below, and in each case provide a brief interpretation of the result.

(a)  $u = (1, 0)$  and  $v = (10, 1)$ .

(b)  $u = (1, 0)$  and  $w = (1, 10)$ .

**Solution.**

(a) We have

$$u \cdot v = 10, \quad \|u\| = 1, \quad \|v\| = \sqrt{101}.$$

Therefore,

$$\text{cosim}(u, v) = \frac{10}{\sqrt{101}} \approx 0.995.$$

Thus the vectors are nearly parallel.

(b) We have

$$u \cdot w = 1, \quad \|u\| = 1, \quad \|w\| = \sqrt{101}.$$

Therefore,

$$\text{cosim}(u, w) = \frac{1}{\sqrt{101}} \approx 0.100.$$

Thus the vectors are nearly perpendicular.

**Question 12. Matrix multiplication (12 points)**

For each of the following matrix multiplications, give the resulting matrix if the multiplication is valid. If the multiplication is not valid, state that it is invalid.

(a)

$$\begin{pmatrix} 1 & 2 & 0 \\ -1 & 3 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & 0 \\ 3 & -1 \end{pmatrix}$$

(b)

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

(c)

$$\begin{pmatrix} 2 \\ -1 \\ 3 \end{pmatrix} (4 \ 5)$$

**Solution.**

(a) This is a  $(2 \times 3)(3 \times 2)$  multiplication, so it is valid and the result has shape  $2 \times 2$ :

$$\begin{pmatrix} 1 & 2 & 0 \\ -1 & 3 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & 0 \\ 3 & -1 \end{pmatrix} = \begin{pmatrix} 4 & 1 \\ 4 & -2 \end{pmatrix}.$$

(b) This is a  $(2 \times 3)(2 \times 2)$  multiplication. It is invalid because the inner dimensions, 3 and 2, are not equal.

(c) This is a  $(3 \times 1)(1 \times 2)$  multiplication, so it is valid and the result has shape  $3 \times 2$ :

$$\begin{pmatrix} 2 \\ -1 \\ 3 \end{pmatrix} \begin{pmatrix} 4 & 5 \end{pmatrix} = \begin{pmatrix} 8 & 10 \\ -4 & -5 \\ 12 & 15 \end{pmatrix}.$$

**Question 13. Matrix multiplication in PyTorch (2 points)**

Fill in the missing PyTorch code with a single operator so that **C** is the matrix product of **A** and **B**.

```

1 A = torch.tensor([
2     [1, 2, 3],
3     [4, 5, 6],
4 ])
5 B = torch.tensor([
6     [1, 2],
7     [3, 4],
8     [5, 6],
9 ])
10 C = A _____ B

```

**Solution.**

`C = A @ B`

Total number of points: ??